Universidad Da Vinci de Guatemala

Facultad de Ingeniería
Estructuras de Datos
Ing. Brandon Chitay

EXAMEN PARCIAL

Lista Doblemente Enlazada Circular

> **Fecha de apertura:** Lunes 13 de abril de 2026, 8:00 AM
> **Fecha de cierre:** Jueves 16 de abril de 2026, 11:59 PM
> **Tiempo estimado de trabajo:** 4 horas
> **Lenguaje:** Java

1. Instrucciones Generales

1. Lea cuidadosamente todas las instrucciones antes de comenzar.
2. Este examen evalúa su capacidad para implementar una Lista Doblemente Enlazada Circular en Java.
3. Tiempo estimado de trabajo: 4 horas. Tiene desde el lunes 13 de abril (8:00 AM) hasta el jueves 16 de abril (11:59 PM) para entregar.
4. [redacted]se permite el uso de inteligencia artificial (ChatGPT, Copilot, Claude, etc.) para generar el código. La evidencia de uso de IA resulta en reprobación automática.
5. Se proporcionan diagramas de flujo y algoritmos paso a paso como guía. Usted debe traducirlos a código Java funcional.
6. El código debe compilar y ejecutarse sin errores.

2. Forma de Entrega

Entregable único: un enlace a un repositorio público de GitHub. El repositorio debe contener:

a) **Código fuente Java** — Todos los archivos .java del proyecto (Nodo.java, ListaDobleCircular.java, Main.java como mínimo).

b) **README.md** — Archivo obligatorio con toda la información requerida (ver sección 3).

c) **Video explicativo** — Enlace a YouTube o Google Drive incluido dentro del README.md (mínimo 5 min, máximo 15 min).

3. Requisitos del README.md

> **IMPORTANTE:** El archivo README.md es TAN IMPORTANTE como el código y el video. Un repositorio sin README.md o con un README.md incompleto resultará en una calificación significativamente menor (-20 puntos). El README.md es la dependencia principal para verificar y evaluar su trabajo.

El README.md debe contener las siguientes secciones:

1. **Información del Estudiante** — Nombre completo, carné, curso, catedrático.
2. **Descripción del Proyecto** — Breve descripción de qué es una lista doblemente enlazada circular y qué operaciones implementó.
3. **Video Explicativo** — Enlace directo al video (YouTube o Google Drive). Sin este enlace no se evalúa el video.
4. **Instrucciones de Compilación y Ejecución** — Comandos exactos para compilar (javac) y ejecutar (java) el proyecto.
5. **Resultados** — Capturas de pantalla o texto copiado de la ejecución del programa mostrando CADA operación funcionando. Sin esta sección no es posible verificar que el programa funciona.
6. **Estructura del Proyecto** — Listado de archivos .java y su propósito.

Ejemplo de estructura del README.md:

```
# Examen Parcial - Estructuras de Datos
## Información del Estudiante
- **Nombre:** [Nombre completo]
- **Carné:** [Número de carné]
- **Curso:** Estructuras de Datos
- **Catedrático:** Ing. Brandon Chitay

## Descripción del Proyecto
[Breve descripción de la implementación]

## Video Explicativo
[Enlace al video]

## Instrucciones de Compilación y Ejecución
```
javac Nodo.java ListaDobleCircular.java Main.java
java Main
```


## Resultados
[Capturas o texto de ejecución aquí]

## Estructura del Proyecto
| Archivo | Descripción |
|---------|-------------|
| Nodo.java | Clase del nodo |
| ListaDobleCircular.java | Operaciones de la lista |
| Main.java | Menú interactivo |
```

4. Descripción del Problema

Una lista doblemente enlazada circular es una estructura de datos en la que cada nodo contiene tres elementos: un dato, una referencia al nodo siguiente y una referencia al nodo anterior. A diferencia de una lista doblemente enlazada convencional, en la versión circular el último nodo apunta de regreso al primer nodo (a través de "siguiente") y el primer nodo apunta al último nodo (a través de "anterior"), formando un ciclo cerrado.

Usted debe implementar esta estructura en Java con las operaciones que se describen en la sección 6. A continuación se muestra la representación visual de la estructura y del nodo.

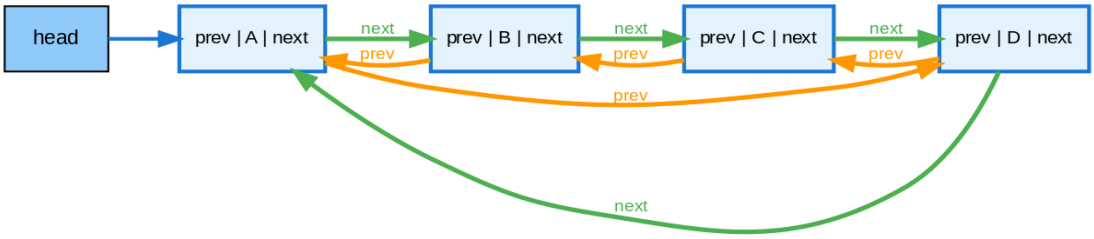

Figura 1: Estructura de una Lista Doblemente Enlazada Circular

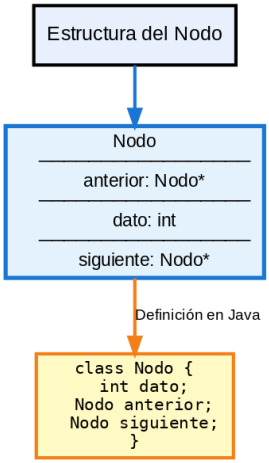

Figura 2: Estructura del Nodo

5. Clase Nodo

Implemente la siguiente clase como punto de partida:

```
public class Nodo {
    int dato;
    Nodo anterior;
    Nodo siguiente;

    public Nodo(int dato) {
        this.dato = dato;
        this.anterior = null;
        this.siguiente = null;
    }
}
```

6. Operaciones a Implementar

6.1 Insertar al Inicio (15 puntos)

Algoritmo paso a paso:

1. Crear un nuevo nodo con el dato proporcionado.
2. Si la lista está vacía (head == null):
   a. nuevoNodo.siguiente = nuevoNodo
   b. nuevoNodo.anterior = nuevoNodo
   c. head = nuevoNodo

Si la lista NO está vacía:
   a. Obtener referencia al último nodo: ultimo = head.anterior
   b. nuevoNodo.siguiente = head
   c. nuevoNodo.anterior = ultimo
   d. ultimo.siguiente = nuevoNodo
   e. head.anterior = nuevoNodo
   f. head = nuevoNodo (el nuevo nodo pasa a ser la cabeza)

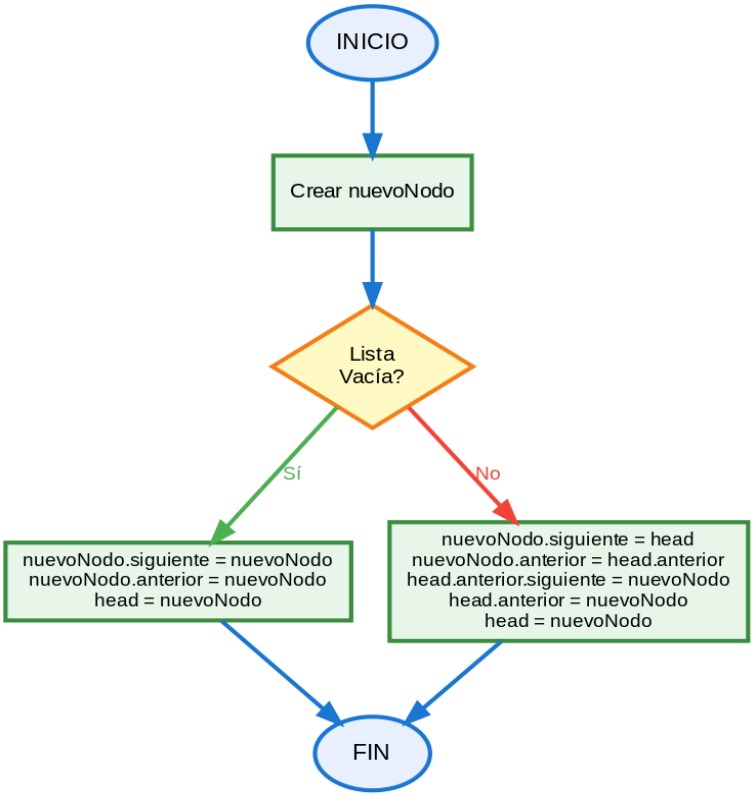

Figura 3: Diagrama de flujo — Insertar al Inicio

6.2 Insertar al Final (15 puntos)

Algoritmo paso a paso:

```
1. Crear un nuevo nodo con el dato proporcionado.
2. Si la lista está vacía (head == null):
     a. nuevoNodo.siguiente = nuevoNodo
     b. nuevoNodo.anterior = nuevoNodo
     c. head = nuevoNodo
3. Si la lista NO está vacía:
     a. Obtener referencia al último: ultimo = head.anterior
     b. nuevoNodo.siguiente = head
     c. nuevoNodo.anterior = ultimo
     d. ultimo.siguiente = nuevoNodo
     e. head.anterior = nuevoNodo
     (Nota: head NO se modifica, a diferencia de insertar al inicio)
```

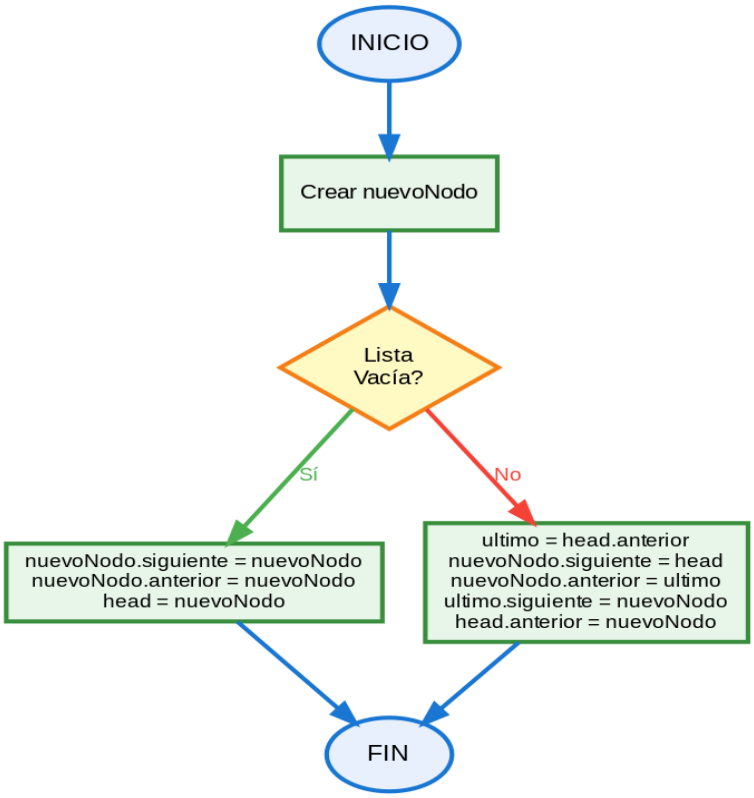

Figura 4: Diagrama de flujo — Insertar al Final

6.3 Eliminar al Inicio (15 puntos)

Algoritmo paso a paso:

```
1. Si la lista está vacía: mostrar error y retornar.
2. Si la lista tiene un solo nodo (head.siguiente == head):
     a. head = null
3. Si la lista tiene más de un nodo:
     a. Obtener referencia al último: ultimo = head.anterior
     b. head = head.siguiente
     c. head.anterior = ultimo
     d. ultimo.siguiente = head
```

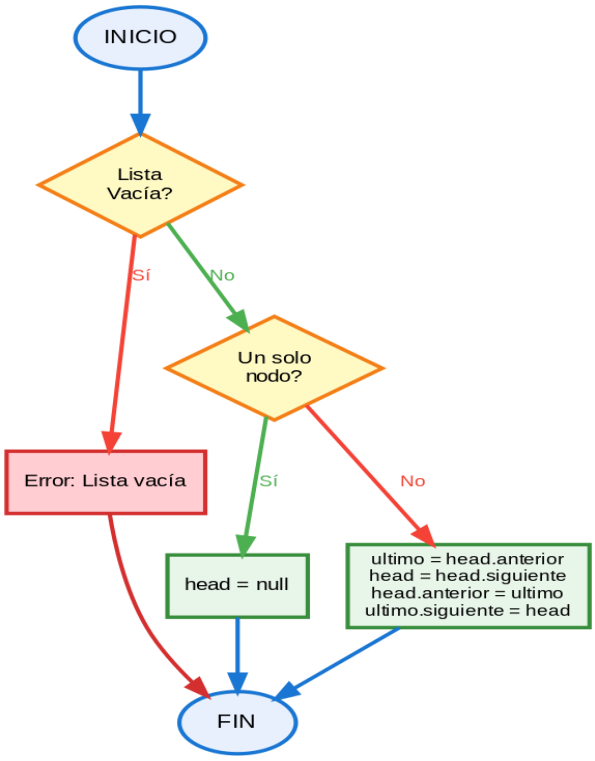

Figura 5: Diagrama de flujo — Eliminar al Inicio

6.4 Eliminar al Final (15 puntos)

Algoritmo paso a paso:

```
1. Si la lista está vacía: mostrar error y retornar.
2. Si la lista tiene un solo nodo (head.siguiente == head):
     a. head = null
3. Si la lista tiene más de un nodo:
     a. Obtener referencia al último: ultimo = head.anterior
     b. Obtener referencia al penúltimo: penultimo = ultimo.anterior
     c. penultimo.siguiente = head
     d. head.anterior = penultimo
```

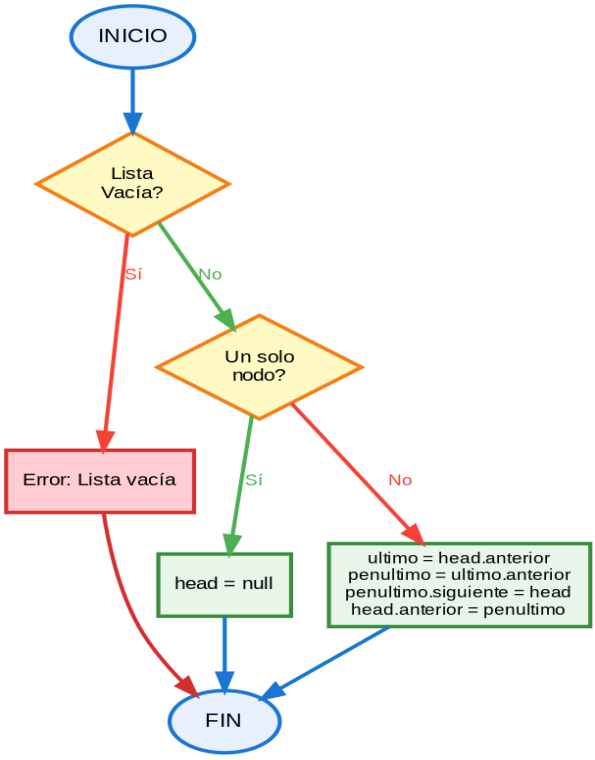

Figura 6: Diagrama de flujo — Eliminar al Final

6.5 Buscar un Elemento (10 puntos)

Algoritmo paso a paso:

```
1. Si la lista está vacía: retornar "No encontrado".
2. Inicializar: actual = head
3. Hacer (do-while):
     a. Si actual.dato == valorBuscado → retornar "Encontrado"
     b. actual = actual.siguiente
4. Mientras actual != head (condición del while)
5. Si se completó el ciclo sin encontrar: retornar "No encontrado".
```

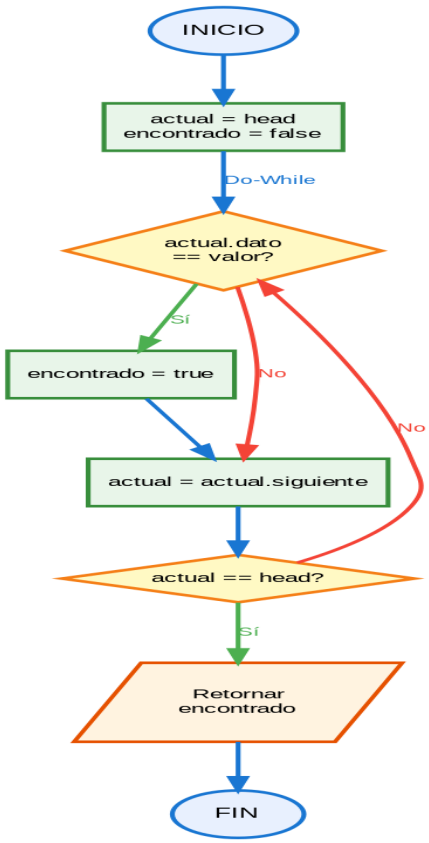

Figura 7: Diagrama de flujo — Buscar Elemento

6.6 Recorrer e Imprimir la Lista (10 puntos — código de referencia)

A continuación se proporciona el código completo de esta operación como referencia. Debe incluirlo en su clase ListaDobleCircular. Le servirá para verificar que las demás operaciones funcionan correctamente.

```
public void imprimir() {
    if (head == null) {
        System.out.println("La lista está vacía");
        return;
    }
    Nodo actual = head;
    System.out.print("Lista: ");
    do {
        System.out.print(actual.dato + " <-> ");
        actual = actual.siguiente;
    } while (actual != head);
    System.out.println("(circular -> " + head.dato + ")");
}
```

6.7 Menú Interactivo (10 puntos)

Cree una clase Main con un método main que muestre un menú repetitivo (usando un ciclo do-while o while) con las siguientes opciones:

```
========================================
  LISTA DOBLEMENTE ENLAZADA CIRCULAR
========================================
1. Insertar al inicio
2. Insertar al final
3. Eliminar al inicio
4. Eliminar al final
5. Buscar elemento
6. Imprimir lista
7. Salir
========================================
Seleccione una opción: _
```

Después de cada operación de inserción o eliminación, el programa debe imprimir automáticamente el estado actual de la lista para que el usuario pueda ver el resultado.

7. Requisitos del Video Explicativo

1. Duración mínima: 5 minutos. Duración máxima: 15 minutos.
2. Debe identificarse verbalmente o mostrar su rostro al inicio del video.
3. Explicar la lógica de cada operación implementada (no solo leer el código, sino explicar qué hace cada parte).
4. Mostrar la ejecución del programa en vivo con al menos 3 operaciones diferentes.
5. El enlace al video debe estar incluido en la sección correspondiente del README.md.

---

8. Criterios de Evaluación

Criterio	Puntos
Insertar al inicio	15
Insertar al final	15
Eliminar al inicio	15
Eliminar al final	15
Buscar elemento	10
Imprimir lista (recorrido)	10
Menú interactivo	10
README.md completo	5
Video explicativo	5
TOTAL	**100**

9. Penalizaciones

Las penalizaciones son requisitos, si estos faltan no se puede determinar que el estudiante haya cumplido o no con el objetivo del examen, que más que un reto tecnico es un reto integral para evaluar las habilidades de documentación, desarrollo y manejo de repositorios del estudiante.

-20 puntos — README.md ausente o incompleto
-15 puntos — Video no entregado o enlace roto
-30 puntos — Código no compila
-10 puntos — Repositorio privado (no público)
-5 pts/hora (máx. -25) — Entrega después de fecha límite
-100 puntos (reprobación) — Evidencia de uso de IA

10. Lista de Cotejo

Use esta lista para verificar que su entrega está completa antes de enviar:

- [ ] El repositorio es público en GitHub
- [ ] Contiene archivo README.md en la raíz del proyecto
- [ ] README.md tiene información del estudiante (nombre, carné)
- [ ] README.md tiene enlace funcional al video
- [ ] README.md tiene sección RESULTADOS con evidencia de ejecución
- [ ] README.md tiene instrucciones de compilación y ejecución
- [ ] El código compila sin errores (javac *.java)
- [ ] Implementa clase Nodo con dato, anterior y siguiente
- [ ] Implementa insertar al inicio correctamente
- [ ] Implementa insertar al final correctamente
- [ ] Implementa eliminar al inicio correctamente
- [ ] Implementa eliminar al final correctamente
- [ ] Implementa búsqueda de elemento
- [ ] Implementa impresión/recorrido de la lista
- [ ] Tiene menú interactivo funcional con todas las opciones
- [ ] El video tiene duración entre 5 y 15 minutos
- [ ] El video muestra la ejecución del programa en vivo